

Segunda Lista de Exercícios - Programação Concorrente

Prof. Gabriel P. Silva - 2026.1

- 1) Qual a diferença entre o valor retornado pelas seguintes rotinas dentro e fora de uma região paralela? (0,5 ponto)
 - a) `omp_get_num_threads()`
 - b) `omp_get_max_threads()`
 - c) `omp_get_num_procs()`
 - d) `omp_in_parallel()`
- 2) Quais as diferenças entre a declaração de variáveis **private**, **firstprivate** e **lastprivate**? Em quais diretivas são utilizadas? (0,5 ponto)
- 3) Quais as diferenças entre as diretivas **master** e **single**? (0,5 ponto)
- 4) Forneça um exemplo do uso da cláusula **reduction** no OpenMP. Descreva quatro tipos de operações de redução que podem ser utilizadas com esta cláusula. (0,5 ponto)
- 5) No código abaixo, faça um diagrama do escalonamento das iterações do laço entre 4 *threads*. (0,5 ponto)

```
#pragma omp for schedule(guided, 5)
    for (j = 0; j < 50; j++)
        a[j] = b[j] + c[j];
```

- 6) Reescreva o código seguinte, colocando as diretivas **barrier** explicitamente, de modo a maximizar o desempenho, mas sem alterar o resultado da execução. (0,5 ponto)

```
#pragma omp parallel
{
    #pragma omp for nowait
        for (j = 0; j < n; j++)
            a[j] = b[j] + c[j];
    #pragma omp for nowait
        for (j = 0; j < n; j++)
            d[j] = e[j] * f;
    #pragma omp for nowait
        for (j = 0; j < n; j++)
```

```

        z[j] = (a[j] + a[j+1]) * 0.5;
    }

```

7) Paralelizar o exemplo a seguir sem o uso da diretiva **for** ou **parallel for**. (0,5 ponto)

```

for (i = 0; i < n; i++)
    z[i] = a * x[i] + y;

```

8) Explique o porquê cada um dos laços a seguir pode ou não pode ser paralelizado com a diretiva **parallel for**. (0,5 ponto)

a) for (i = 0; i < N ; i ++)
 if (x [i] > maxval) break;

b) for (i = 0; i < N ; i ++)
 for (j = 0; j < i ; j++)
 a[j][i] = a[j+1, i];

c) for (k = 0 ; k < N ; k++)
 x[k] = q + y[k]*(r*z[k+10] + t*z[k+11]);

d) for (i = 1 ; i < N ; i++)
 x[i] = z[i]*(y[i] - x[i-1]);

e) for (i = 0; i < N ; i ++) {
 a[i] = a[i] * a[i];
 if (fabs(a[i]) > machine_max || fabs(a[i]) <
machine_min){
 printf ("i = %d \n", i);
 break;
 }
 }

f) for (i = 1 ; i < N ; i++)
 for (k = 0 ; k < i ; k++)
 w[i] += b[k][i] * w[(i-k)-1];

g) for (k = 0 ; k < N ; k++)
 x[k] = u[k] + r*(z[k] + r*y[k]) +
 t*(u[k+3] + r*(u[k+2] + r*u[k+1]) +

```
t*( u[k+6] + r*( u[k+5] + r*u[k+4] ) ) );
```

9) Considere o seguinte laço: (0,5 ponto)

```
x = 1;
#pragma omp parallel for firstprivate (x)
for(i = 0; i < N; i++){
    y [i] = x + i;
    x = i;
}
```

- a) Porque este laço está incorreto? $y[i]$ recebe o mesmo resultado independente do número de *threads* executando o laço?
- b) Qual o valor da variável 'i' ao final do laço? Qual é o valor da variável 'x' ao final do laço?
- c) Qual seria o valor de 'x' ao final do laço se seu escopo fosse **shared**?
- d) Este laço pode ser paralelizado corretamente (isto é, preservando a semântica sequencial) apenas com o uso de diretivas OpenMP?

10) Quais os mecanismos disponíveis no OpenMP para lidar com as condições de corrida? (0,5 ponto)

11) Descreva como a diretiva **ordered** é utilizada no OpenMP. Apresente um trecho de código como exemplo. (0,5 ponto)

12) Como a cláusula **collapse** funciona no OpenMP e em que situações deve-se utilizá-la? (0,5 ponto)

13) Explique a diferença fundamental entre tarefas explícitas e tarefas implícitas no OpenMP. Em que contextos cada tipo de tarefa é gerado? (0,5 ponto)

14) Defina o que é um ponto de escalonamento de tarefas. Liste pelo menos três ações distintas que uma *thread* pode tomar ao encontrar um ponto de escalonamento durante a execução de uma tarefa. (0,5 ponto)

15) O que são os pontos de sincronização de tarefas, e como a diretiva **#pragma omp taskwait** é especificamente utilizada neste contexto? (0,5 ponto)

16) No trecho de código a seguir, identifique adequadamente os pontos de escalonamento e de sincronização de código. (1,0 ponto)

```
#include <stdio.h>
#include <omp.h>
int main() {
```

```

#pragma omp parallel num_threads(4)
{
    #pragma omp single
    {
        #pragma omp task
        printf("Executando Tarefa A \n");

        #pragma omp taskyield
        printf("Após taskyield \n");

        #pragma omp task
        printf("Executando Tarefa B \n");

        #pragma omp taskwait
        printf("Após taskwait \n");

        int data = 0;
        #pragma omp task depend(out: data)
        data = 42;

        #pragma omp task depend(in: data)
        printf("Data = %d\n", data);

        #pragma omp taskgroup
        {
            #pragma omp task
            printf("Tarefa E no taskgroup \n");
            #pragma omp task
            printf("Tarefa F no taskgroup \n");
        } // Fim do taskgroup
    } // Fim do single
} // Fim do parallel
return 0;
} // Fim do programa

```

- 17) Descreva a função das cláusulas **if e final** e como elas se diferenciam na geração e execução das tarefas. (0,5 ponto)

- 18) Considere o seguinte trecho de código. Aplique a cláusula **depend** com os argumentos corretos para garantir a execução correta das tarefas. Em seguida, desenhe o grafo de dependências. (0,5 ponto)

```
int main() {
    int A = 0, B = 0, C = 0;
    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp task        // Tarefa A
            A = 1;
            #pragma omp task        // Tarefa B
            B = A + 1;
            #pragma omp task        // Tarefa C
            C = B + C;
            #pragma omp task        // Tarefa D
            printf("Resultado final %d %d %d \n", A,B, C);
        } // Fim do single
    } // Fim do parallel
    return 0;
} // Fim do programa principal
```

- 19) Desenhe o grafo de dependências para o trecho de código a seguir: (1,0 ponto)

```
int main() {
    int x = 0, y = 0, z = 0;
    #pragma omp parallel
    {
        #pragma omp single
        {
            // Task 1:
            #pragma omp task depend(out: x) shared(x)
            x = 5;
            // Task 2:
            #pragma omp task depend(in: x, y, z) shared(x, y, z)
            int resultado = x + y + z;
            // Task 3:
            #pragma omp task depend(in: x) depend(out: y) shared(x, y)
```

```

        y = x * 2;
    // Task 4:
    #pragma omp task depend(in: x) depend(out: z) shared(x, z)
        z = x + 10;
    // Task 5:
    #pragma omp task depend(inout: y) shared(y)
        y = y + 3;
    // Task 6:
    #pragma omp task depend(inout: z) shared(z)
        z = z - 2;
    }
}
printf("Valores finais: x = %d, y = %d, z = %d\n", x, y, z);
return 0;
}

```

- 20) Explique a diretiva **#pragma omp taskloop**. Mencione pelo menos duas cláusulas específicas que podem ser usadas com **taskloop** e descreva suas funções. (0,5 ponto)